

AWS Migration Plan

Supabase → RDS · Railway → ECS Fargate · Resend → SES · Auth: Self-Hosted GoTrue on ECS · New NestJS Backend

Document	Migration & HIPAA Compliance Plan
Version	1.1 — auth revised from AWS Cognito to self-hosted GoTrue
Date	July 15, 2026
Status	Approved
Scope	myclinicmd EMR (Next.js 14 monolith) — full platform migration to AWS
Timeline	~4–6.5 months, 1–2 engineers (~24–33 engineer-weeks)
Compliance milestone	All PHI at rest + auth under AWS BAA by ~month 3



Contents

1. Context & Motivation
2. Locked Decisions
3. Verified Current State
4. End-State Architecture
5. Sequencing Strategy
6. Phase 0 — AWS Foundation, Docker, CI/CD, Hosting → ECS
7. Phase 1 — Resend → SES
8. Phase 2 — Storage → S3
9. Phase 3 — Database + Auth → RDS via PostgREST Shim + Self-Hosted GoTrue
10. Phase 4 — GoTrue Hardening & Supabase Decommission
11. Phase 5 — NestJS Backend & Incremental Domain Port
12. Phase 6 — Shim Retirement & HIPAA Hardening
13. What Does Not Change
14. Risk Register
15. Effort & Compliance Timeline
16. Verification Strategy

1. Context & Motivation

The driver is HIPAA. MEMR (myclinicmd) is a production medical EMR handling Protected Health Information (PHI). The current vendor stack either charges heavily for Business Associate Agreements or offers none:

- **Supabase:** BAA requires Team plan + HIPAA add-on ≈ **\$950/month**
- **Railway:** BAA requires Enterprise track ≈ **\$1,000/month minimum**
- **Resend:** no BAA offering at all

The decision: consolidate the entire platform under the **single free AWS BAA** (RDS, ECS Fargate, S3, SES, CloudFront, ElastiCache, Lambda, and Textract are all HIPAA-eligible services), adopting a NestJS backend as the target architecture. Because Supabase remains BAA-uncovered during the transition, **PHI-heavy components move as early as possible.**

Parallel obligation: PHI also flows to **OpenAI** (SOAP generation + 9 AI features) and **Daily.co** (video visits + transcription). An OpenAI API BAA (free, on request) and a Daily.co HIPAA plan must be signed regardless of this migration — the paperwork workstream opens in Phase 0. The migration is not compliance-complete without them.

2. Locked Decisions

Area	From	To
Database	Supabase Postgres	AWS RDS PostgreSQL (Multi-AZ, KMS, PITR 35d)
Hosting	Railway	ECS Fargate (Next.js web + NestJS api), ALB, CloudFront
Email	Resend	AWS SES
Auth	Supabase-hosted GoTrue	Self-hosted GoTrue on ECS (rev. 1.1 — replaces Cognito; zero app-code auth rewrite, passwords/IDs/sessions preserved; Cognito remains a later option)
File storage	Supabase Storage (7 buckets)	S3 presigned URLs, KMS, CloudFront OAC for assets
Backend	Next.js API routes (~110 handlers)	NestJS service, ported incrementally
IaC	None	Terraform (greenfield)
Strategy	—	Incremental strangler — always shippable, no big-bang

The Next.js frontend stays the same codebase — it is SSR (cannot be static-hosted), so it runs as a container on ECS with CloudFront in front. Pages, components, and styling are untouched.

3. Verified Current State

Findings from a full codebase exploration (three independent mapping passes):

- Next.js 14.2 App Router **SSR monolith**; ~110 API route handlers in `app/api/**` (the real data layer), domain logic in `lib/**`; `middleware.ts` handles auth/role routing. Production start must be `node scripts/next-start.cjs`.
- **All DB access is server-side**: ~430 `.from('table')` PostgreSQL call sites across ~120 files; **zero** browser data queries; **zero** `.rpc()` calls; no generated DB types. Clients centralized in `lib/supabase/{client,server,admin,user-jwt-client,keys}.ts`.
- Auth: **email/password only** (no OAuth, MFA, or self-service reset); server-side login route with Cloudflare Turnstile + rate limiting; **roles come from the profiles table, not JWT claims** (`lib/fetch-user-role.ts`, `lib/roles.ts`: ADMIN, DOCTOR, FNP, PA, NURSE + staff).
- Schema: ~50 tables, 111 SQL migrations, **284 RLS policies**, ~25 plpgsql functions/triggers (two SECURITY DEFINER triggers on `auth.users` auto-create profiles), **no Postgres extensions** (no pgvector, no pg_cron).
- Storage: 7 buckets (3 PHI-private with signed URLs). Realtime: only 3 files (chat + support). Edge functions: only 2, neither invoked from app code (QR-registration webhook + legacy SOAP forwarder).
- Resend: exactly **one** email flow (SOAP-note-to-patient, `lib/email/*`). AWS Textract already in use (static keys → will switch to task roles).
- No Dockerfile, no IaC, no deploy CI — Railway GitHub integration only. Branch mapping: `qa` = staging, `raheel` = production/default.

Pre-cutover check: Supabase project-ref mismatch must be resolved — `supabase/config.toml` references `kxtbfmlatysaxjzppmti` (production) while the local `.env` points at `bssjlpqdgmtgaxvadwdh` (staging). Confirm which project each runtime environment actually uses before any data migration.

4. End-State Architecture

CloudFront → **ALB** → **web** (Next.js, ECS Fargate) + **api** (NestJS, ECS Fargate, path-routed `/api/v2/*`) + **gotrue** (self-hosted Supabase Auth, internal ECS service, path-routed `/auth/v1/*`) → **RDS Postgres** (Multi-AZ, KMS, PITR 35d) + **ElastiCache Redis** + **S3** (KMS; private PHI bucket + assets bucket via CloudFront OAC)

SES email · Secrets Manager / SSM config · GitHub Actions (OIDC) → ECR → ECS · Two environments: staging (`qa` branch) / production (`raheel` branch). GoTrue is a permanent component (pinned image, same JWT secret as the PostgREST shim); NestJS verifies GoTrue JWTs.

5. Sequencing Strategy

The pivotal insight: the app speaks pure PostgREST through centralized client factories, so **the database moves to RDS early (Phase 3) behind a self-hosted PostgREST shim** — the ~430 query call sites don't change at all, RLS keeps working (GoTrue JWTs verify against the shim), and the largest PHI store is BAA-covered by ~month 3 instead of at the end of the project. The scariest step (data cutover) happens early, small, and rehearsed — and the NestJS port then develops against RDS from day one.

Gate: a 1-week spike precedes Phase 3 — staging dump (public + auth schemas) restored into RDS + official PostgREST + **self-hosted GoTrue (pinned to the hosted project's version)** + facade client + jest/manual smoke of the top 10 routes including login, getUser, admin user-create, and password change. If the spike fails, fall back to DB-last ordering; every other phase is order-compatible.

#	Phase	Calendar	BAA gap closed at completion
0	AWS foundation, Docker, CI/CD, hosting → ECS	Wks 1–4	Compute/hosting
1	Resend → SES	Wk 4–5	PHI-bearing SOAP patient emails
2	Supabase Storage → S3	Wks 5–7	Patient documents, passports, consent forms, attachments
3	Postgres + Auth → RDS (PostgREST shim + self-hosted GoTrue)	Wks 7–13	The entire database — all table PHI — and staff credentials
4	GoTrue hardening + Supabase decommission	Wks 13–15	Supabase project deleted (after 30-day safety window)
5	NestJS backend + domain port + WebSockets	Wks 17–34	(architecture end-state; no new BAA delta)
6	Shim retirement, scale-out, HIPAA hardening	Wks 34–37	Residual hardening; OpenAI/Daily BAAs confirmed

6. Phase 0 — AWS Foundation, Docker, CI/CD, Hosting → ECS

Weeks 1–4 · 3–4 engineer-weeks

- **Terraform greenfield:** `infra/global` (ECR, GitHub-OIDC IAM, CloudTrail) + `infra/modules/` (network, ecs-cluster, service, rds, elasticache, postgres, gotrue, cloudfront, ses, s3-storage, waf, observability) + `infra/envs/{staging,production}` with separate state. VPC 2-AZ (public ALB/NAT, private ECS, isolated DB subnets), ECS cluster, ALB, CloudFront + ACM, Secrets Manager, CloudWatch.
- **Dockerfile** (multi-stage, parity-first non-standalone; entrypoint `node scripts/next-start.cjs`; output: `'standalone'` deferred as an optimization) + `.dockerignore`.
- **CI/CD:** extend `.github/workflows/ci.yml` with deploy jobs — `qa` → staging, `raheel` → production. Per-environment image builds (build args `APP_ENV`, `NEXT_PUBLIC_*` — `APP_ENV` is baked into the client bundle). Sentry sourcemaps upload inside the image build.

- Env vars → Secrets Manager/SSM; Textract switched to the ECS task role (`lib/i693/supporting-documents/textract.ts` → default credential chain; static AWS keys deleted).
- **Run 1 task per service** — `lib/rate-limit.ts` / `lib/cache.ts` are in-memory; Redis arrives in Phase 5. ALB health check = existing `GET /api/health`.
- DNS → CloudFront (Cloudflare grey-cloud if applicable). Railway kept hot 2 weeks as rollback (rollback = DNS flip back).
- **Open the BAA paperwork workstream: OpenAI API BAA + Daily.co HIPAA plan.**

Verification: staging smoke (login, encounter flow, chat, document upload, video, SOAP email); health green; Sentry/PostHog flowing; ≥3-day staging soak before the production DNS flip.

BAA delta: application compute on Fargate under the AWS BAA (Railway had none — a real gap closure).

7. Phase 1 — Resend → SES

Week 4–5 · 0.5–1 engineer-weeks

- Terraform: SES domain identity `alerts.myclinicmd.com`, Easy DKIM, configuration set + bounce/complaint alarms. **File the SES production-access request on day 1** (manual approval, calendar lead time).
- Swap the internals of `lib/email/resend-client.ts` to `@aws-sdk/client-sesv2`, preserving the exported `sendEmail()` signature so all callers are untouched. SES MessageId stored in the existing `resend_message_id` audit column.
- `EMAIL_PROVIDER=ses|resend` flag for 2 weeks of instant rollback, then remove the `resend` dependency.

Verification: staging sends; DKIM/SPF/DMARC pass; audit row written; bounce alarm fires.

BAA delta: SOAP-note patient emails (PHI in transit through the email provider) under the AWS BAA.

8. Phase 2 — Storage → S3

Weeks 5–7 · 1.5–2 engineer-weeks

- Terraform: `mcm-{env}-phi` bucket (SSE-KMS, versioned, block-public, TLS-only policy) with prefixes mirroring the six private Supabase buckets; `mcm-{env}-assets` for `staff-avatars` served via CloudFront OAC.
- New `lib/storage/s3.ts` (put / presigned get & put / delete / list + bucket→{bucket, prefix} map that **preserves existing object keys so DB path columns don't change**). Swap the ~11 `.storage` call sites. Audit for columns storing full Supabase URLs (avatars likely) → one-off rewrite script.
- Data copy: `rc1one sync` per bucket via Supabase's S3-compatible endpoint; verify object counts, total bytes, spot checksums. Update CSP + `images.remotePatterns` in `next.config.js`.

Verification: upload/download/delete round-trips for every document class; historical documents open from real patient records; signed-URL expiry semantics match today's behavior.

BAA delta: all PHI documents at rest under the AWS BAA. Remaining in Supabase: Postgres, Auth, Realtime.

9. Phase 3 — Database + Auth → RDS via PostgREST Shim + Self-Hosted GoTrue ★

Weeks 7–13 · 4–6 engineer-weeks · the pivotal phase

- **Infra:** RDS Postgres (match Supabase's major version; Multi-AZ, KMS, PITR 35d, `rds.force_ssl=1`, deletion protection, isolated subnets). Internal ECS service `postgrest` (official image; `PGRST_JWT_SECRET` = the Supabase legacy JWT secret; never internet-exposed). Internal ECS service `gotrue` (supabase/auth image **pinned to the version the hosted project runs**; same JWT secret; `GOTRUE_DISABLE_SIGNUP=true`; SES SMTP wiring). ALB path routing on the app domain: `/auth/v1/*` → `gotrue` — mirroring Supabase's Kong routing so supabase-js URLs work unchanged.
- **RDS prep:** roles `anon/authenticated/service_role`. **The full auth schema travels in the dump** (auth.users including bcrypt password hashes, refresh tokens, GoTrue's own tables) — the `profiles` FK, both profile-creation triggers, and all 284 RLS policies work verbatim with **no mirror table, no dual-writes, no trigger replacement**. Schema source of truth = `pg_dump --schema-only` of **live production** (guards against migration-file drift), diffed against a migrations-built schema as a verification artifact. `scripts/apply-supabase-migration.cjs` repointed at RDS + `NOTIFY pgrst, 'reload schema'`.
- **Code:** `lib/supabase/{admin,server,user-jwt-client}.ts` become facades — `from()` delegates to `@supabase/postgrest-js` at `DATA_API_URL`; `auth.*` is supabase-js pointed at the app domain (self-hosted GoTrue), so the login route (Turnstile + rate limiting), `middleware.ts`, the `admin.auth.admin.*` calls, all ~85 `getUser()` sites, and the browser auth contexts **keep working unchanged** — only `NEXT_PUBLIC_SUPABASE_URL` flips. (The `sb-*` cookie name derives from the URL, so staff re-log-in once at cutover.) The three Realtime files convert to polling (chat 3–5s, support 10–15s; temporary until the Phase 5 WebSocket gateway). Edge function `insert-appointment-with-dob-check` ports to a Next route preserving the `X-API-KEY` contract; `intake-form-soap-ts` retired (confirm with owner).
- **Cutover runbook** (rehearsed ≥2× on staging; project-ref mismatch resolved first): overnight window 2–4h → maintenance page → `pg_dump -Fc` (public + auth) → restore → verify (row counts; PK-ordered md5 checksums on patients, encounters, prescriptions, audit_logs, profiles, auth.users; sequences; business spot checks) → flip `DATA_API_URL` + `NEXT_PUBLIC_SUPABASE_URL` → smoke including login → reopen → 48h elevated monitoring. In-window rollback = env flip (Supabase untouched); Supabase kept idle 30 days.

BAA delta: the entire relational PHI store *and* staff credentials on RDS under the AWS BAA. Nothing PHI-bearing remains in Supabase.

10. Phase 4 — GoTrue Hardening + Supabase Decommission

Weeks 13–15 · 0.5–1 engineer-weeks

Replaces the original Cognito migration (decision revised July 15, 2026). Because the self-hosted GoTrue went live with the Phase 3 cutover, there is no auth rewrite: no migration Lambda, no ID re-mapping, no new session model. Cognito remains a documented later option if fully managed auth is ever wanted.

- **Harden the now-live GoTrue service:** version pinning + upgrade runbook (subscribe to supabase/auth releases — CVE patching is owned in-house now), CloudWatch alarms on `/auth/v1/*` 5xx and latency,

rotation runbook for `GOTRUE_JWT_SECRET` (coordinated with the PostgREST shim and facade), backup verification of the `auth` schema within RDS PITR, GoTrue upgrade drill on staging.

- **Net-new (optional, small): self-service forgot-password** via `supabase-js resetPasswordForEmail` → GoTrue `/recover` → SES SMTP — the first self-service reset the product has had; admin-set passwords remain the fallback.
- After the 30-day safety window: **the Supabase project is deleted — the recurring-fee problem is fully solved here.** `*.supabase.co` purged from CSP and `images.remotePatterns`.

Verification: role×route matrix (ADMIN/DOCTOR/FNP/PA/NURSE × middleware rules + API guards); fail-closed checks; session expiry/refresh; staging soak with real staff accounts; upgrade drill completed.

BAA delta: Supabase gone entirely — all vendors with recurring BAA fees eliminated.

11. Phase 5 — NestJS Backend + Incremental Domain Port + WebSockets

Weeks 17–34 · 12–16 engineer-weeks · parallelizes across 2 engineers

- New `apps/api/` in the same repo (own `package.json`, `Dockerfile.api`); ECS service `api`; **same-origin path routing** `/api/v2/*` → zero CORS/cookie-domain issues. Auth guard verifies GoTrue JWTs (shared legacy HS256 secret, same as the PostgREST shim; optional later move to asymmetric keys).
- **ORM: Kysely + kysely-codegen** — introspects RDS into generated types; the chainable builder is a near-1:1 mental model with the PostgREST chains, making the port mechanical; raw SQL migrations remain the schema-change mechanism (Prisma/TypeORM would fight the 111-file SQL corpus).
- Skeleton: `AuthModule`; `CommonModule` with `RolesGuard` + `@Roles()` (replicating `lib/security/api-auth.ts`), `LocationScopeService` (replicating `user_assigned_location_ids()` /tenant scoping — the app-layer RLS replacement), and an audit interceptor writing the same `audit_logs` shape; `HealthModule`. **ElastiCache Redis lands here** (Socket.IO Redis adapter, throttler, cache) — unblocking horizontal scale-out.
- **Port recipe per domain:** enumerate routes + tables → capture golden request/response fixtures from the legacy route → build module → translate PostgREST→Kysely (watch `maybeSingle`, `count:'exact'`, range pagination, null-vs-empty semantics) → guards + audit → contract-test against goldens → flip frontend to `/api/v2/...` → monitor → delete legacy route. DB triggers/generators need no porting — they live in RDS.
- **Port order** (simple → hub): health → locations/tenants → pharmacies → patients → vitals → appointments → **encounters** → soap → prescriptions → i693/immigration → compliance → **chat + support with the Socket.IO gateway replacing polling** → admin → dashboard. Burn-down metric: `grep count` of remaining `.from()`.

Verification per domain: golden contract tests green; role-matrix guard tests; audit rows identical in shape; staging canary before each production flip.

12. Phase 6 — Shim Retirement + HIPAA Hardening

Weeks 34–37 · 2–3 engineer-weeks

- When the `.from()` count reaches zero: delete the postgrest ECS service, `@supabase/postgrest-js`, and the data half of the `lib/supabase/` facade (the auth half — `supabase-js` against self-hosted GoTrue — is

permanent; the `auth` schema stays live in RDS). Scale web/api to ≥ 2 tasks across AZs; add GoTrue to the patch/upgrade cadence.

- Hardening: GuardDuty; AWS Config HIPAA conformance pack (optional); AWS Backup + cross-region RDS snapshot copies; audit-log export to S3 Glacier (≥ 6 -year retention); ALB/CloudFront access logs; WAF managed rules on CloudFront; incident-response and restore-drill runbooks; quarterly IAM access review; Sentry/PostHog PII-scrub review; BetterStack log forwarding retired (CloudWatch native).
- **Confirm the OpenAI BAA and Daily.co HIPAA plan are signed — the migration is not compliance-complete without them.**

13. What Does Not Change

All UI pages and components; middleware role-routing rules; Daily.co, OpenAI, Smarty, Mapbox, Turnstile, PostHog, Sentry integrations (scrub-config review only); Zod validation; the I-693 PDF pipeline (mupdf/pdf-lib); the DB schema and its plpgsql functions/triggers (they travel as-is); the `qa / rahee1` branch process; the jest suite; all `lib/**` business semantics — only the transport layer swaps underneath them.

14. Risk Register

Risk	Mitigation
PostgREST shim behavioral drift vs Supabase-hosted	1-week spike gate; pin same version; golden contract tests on top-20 routes; explicit fallback to DB-last ordering
Migration files $\neq$ live production schema	<code>pg_dump --schema-only</code> of live prod is the source of truth; diff report as artifact
Cutover data loss / window overrun	≥ 2 timed staging rehearsals; scripted runbook; count/checksum gates block the flip; Supabase retained idle 30 days
Self-hosted GoTrue ops burden (CVEs, upgrades, supabase-js compatibility drift)	Pin image to the hosted project's version; upgrade runbook + staging drill per release; alarms on <code>/auth/v1 5xx</code> ; Cognito documented as escape hatch
GoTrue-on-RDS incompatibility (schema ownership, migrations)	Covered by the extended Phase 3 spike gate — <code>login/getUser/admin-create/password-change</code> verified before committing to the ordering
Session regression locks staff out — or fails open	Fail-closed defaults; explicit per-role matrix tests; staged rollout
In-memory rate-limit/cache breaks silently at > 1 ECS task	Hard rule: <code>desired-count = 1</code> until the Redis swap ships (Phase 5); alarm on task count
SES sandbox/deliverability for PHI emails	Production-access request on day 1; DKIM/SPF/DMARC verified in staging; bounce alarms; provider rollback flag
PostgREST $\rightarrow$ Kysely semantic bugs across 430 sites	Written recipe + translation cheat-sheet; golden fixtures per route; per-domain canary flips; <code>grep</code> burn-down gate before shim deletion
Interim chat/support polling degrades UX	Short window (Phase 3 $\rightarrow$ 5); conservative intervals with backoff; WebSocket gateway prioritized early in the Phase 5 port order
OpenAI / Daily.co still receive PHI without BAAs	Paperwork workstream opened in Phase 0 with an owner and due date; tracked to signature

15. Effort & Compliance Timeline

Phase	Engineer-weeks	Notes
0 — Foundation & hosting	3–4	Terraform foundation is the bulk; Docker/CI ~1 ew
1 — SES	0.5–1	One email flow; SES approval is calendar time, not effort
2 — S3	1.5–2	~11 call sites + rclone copy + verification
3 — RDS + shim	4–6	Spike, shim + facade, runbook, 2 rehearsals, cutover
4 — GoTrue hardening + decommission	0.5–1	Upgrade runbook, alarms, optional forgot-password; no auth rewrite (GoTrue decision saves ~3 ew vs Cognito)
5 — NestJS port	12–16	~15 domains × 0.7–1.2 ew; parallelizes across 2 engineers
6 — Hardening	2–3	Retirement + hardening + drills
Total	~24–33	≈ 4–6.5 calendar months at 1–2 engineers

Compliance milestones: all PHI at rest (database + documents), staff credentials, and PHI-bearing email under the AWS BAA by ~**month 3**; the Supabase project is deleted after the 30-day safety window (~**month 3.5**); the long NestJS port proceeds afterward without compliance pressure.

16. Verification Strategy

- Every phase ends with a staging soak on qa plus a manual smoke of the clinical golden path: login → open encounter → vitals → SOAP generate → document upload → SOAP email → chat.
- Role×route authorization matrix re-run after any auth or guard change.
- Data-integrity gates (row counts, PK-ordered checksums) must pass before any cutover flip.
- 48 hours of elevated monitoring after every production change.